■ MediBot System Architecture & Code Flow

Enterprise Medical Assistant with Database-Layer RBAC, Hybrid Search & SQL RAG

MediAssist Health Network

1. Executive Summary

MediBot is an enterprise medical and operational AI assistant built for MediAssist Health Network (12 hospitals and 40+ clinics in India). It fulfills two mission-critical requirements: (1) high-precision retrieval across medical protocols, drug formularies, nursing standards, equipment manuals, and insurance tariffs; and (2) zero-trust Role-Based Access Control (RBAC) enforced directly at the vector database retrieval level to eliminate data leakage even under adversarial jailbreaks.

2. End-to-End System Flow Diagram

```
+-----------------------------------------------------------------------------------------+ | Staff User (Doctor / Nurse / Billing / Technician / Admin) | | POST /chat { question, role, auth_token } | +-----------------------------------------+-----------------------------------------+ | v [ FastAPI Backend Gateway (backend/main.py) ] | +--------------------+--------------------+ | 1. Security Guardrails & Verification | | • Gibberish filter & API-key protection| +--------------------+--------------------+ | [ Intent Classifier ] / \ (Analytical/Claims Question) (Policy/Clinical Manual Question) / \ v v +-------------------------------+ +-------------------------------+ | BRANCH A: SQL RAG ENGINE | | BRANCH B: HYBRID DOCUMENT RAG | | (backend/rag/sql_rag.py) | | (backend/rag/hybrid_retriever) | +-------------------------------+ +-------------------------------+ | 1. RBAC Perms Check | | 1. Collection RBAC Pre-Check | | (Billing Exec / Admin only) | | 2. Qdrant Vector Filter | | 2. NL-to-SQL Generation (LLM) | | must=[access_roles == role] | | 3. Safety Sanitizer (SELECT) | | 3. Dual Dense + Sparse Search | | 4. Query SQLite mediassist.db | | 4. Reciprocal Rank Fusion (RRF)| | 5. LLM Answer Synthesis | | 5. Cross-Encoder Rerank (Top 3)| +---------------+---------------+ +---------------+---------------+ | 6. Grounded LLM Synthesis | | +---------------+---------------+ \ / \--------------------+---------------------/ v [ Structured JSON Response Returned to UI ] • Grounded Answer text with citations & badges • Sources: [{ document, section, score }] • Security Guardrail & RBAC violation flags
+-----------------------------------------------------------------------------------------+
```

3. Phase 1: Ingestion Pipeline (Data Preparation)

• **Document Parsing (backend/ingestion/parser.py):** Parses PDFs and Markdown files preserving headings, bullet points, and tables. Attaches hierarchical metadata breadcrumbs like *[Document > Section > Subsection]*.
• **Role Tagging:** Every chunk is tagged with permissible access roles (e.g., clinical guides get *doctor, admin*; tariffs get *billing_executive, admin*).
• **Dual Embeddings (backend/ingestion/embedder.py):** Dense embeddings (384d, *bge-small-en-v1.5*) for semantic matching plus Sparse BM25 embeddings (*Qdrant/bm25*) for medical keywords.
• **Vector Database (backend/ingestion/index_qdrant.py):** Upserts points into Qdrant collection *mediassist_knowledge* with dedicated payload indexes.

4. Phase 2: Step-by-Step Runtime Code Flow

Step 1: Request Gateway & Pre-Flight Guardrails

FastAPI receives the request at /chat. It checks for gibberish patterns (vowel ratios, character repetitions) and blocks malicious prompt injections attempting to extract system API keys.

Step 2: Intent Classification & Routing

Identifies if the query is *Analytical / Statistical* (contains terms like count, claims, sum, average, tickets) or *Document-based* (treatment guidelines, dosages, manuals).

Branch A: SQL RAG Engine (Analytical Queries)

1. **RBAC Check:** Restricted to billing_executive and admin. Unauthorized roles receive a clear refusal.
2. **NL to SQL:** LLM converts the request into standard SQLite targeting claims or maintenance_tickets.
3. **SQL Sanitizer:** Enforces read-only safety (must start with SELECT; drops DDL/DML like DROP, DELETE, UPDATE).
4. **Execution & Synthesis:** Queries mediassist.db and uses the LLM to summarize the returned records in natural language.

Branch B: Hybrid Document RAG (Policies & Manuals)

1. **Domain Pre-Check:** Rejects queries explicitly asking for collections outside the user's scope.
2. **Database-Level RBAC Filter:** Enforced inside the Qdrant retrieval query (must=[FieldCondition(key='access_roles', match=MatchValue(value=role))]). Restricted chunks are never retrieved into memory.
3. **Hybrid Search & RRF:** Qdrant evaluates dense and sparse BM25 vectors simultaneously and fuses candidate chunks using Reciprocal Rank Fusion (RRF) to retrieve the top 10 candidates.
4. **Cross-Encoder Reranking:** cross-encoder/ms-marco-MiniLM-L-6-v2 jointly re-scores the query and candidates to pick the top 3 chunks.
5. **LLM Grounded Synthesis:** LLM synthesizes an answer referencing only verified citations.

5. User Roles & Access Control Matrix

Role	Department	Accessible Collections	SQL RAG?	Demo User
doctor	Clinical Cardiology	clinical, nursing, general	■ No	dr.mehta
nurse	ICU & Patient Care	nursing, general	■ No	nurse.priya
billing_executive	Billing & Insurance	billing, general	■ Yes	billing.ravi
technician	Biomedical Eng.	equipment, general	■ No	tech.anand
admin	Executive / IT	All 5 collections	■ Yes	admin.sys

6. Key Architecture Components & Source Files

Module File	Layer	Functionality
backend/main.py	API Gateway	FastAPI endpoints (/chat, /login, /collections, /health).
backend/ingestion/parser.py	Ingestion	Parses PDFs and Markdown, builds hierarchical context & chunks.
backend/ingestion/embedder.py	Embedding	Generates dense (bge-small-en) & sparse (BM25) vector representations.
backend/ingestion/index_qdrant.py	Vector Storage	Creates collections, defines schemas, and upserts chunk points.
backend/rag/hybrid_retriever.py	Retrieval Core	Executes Qdrant RBAC filtering, RRF fusion, and pipeline coordination.
backend/rag/reranker.py	Reranking	Cross-Encoder reranking using ms-marco-MiniLM-L-6-v2 down to Top 3.
backend/rag/sql_rag.py	SQL Analytics	NL-to-SQL generation, query sanitizer, SQLite execution on claims/tickets.
backend/rag/llm_service.py	LLM Inference	Multi-model client supporting Gemini, Groq, OpenAI & local fallback.